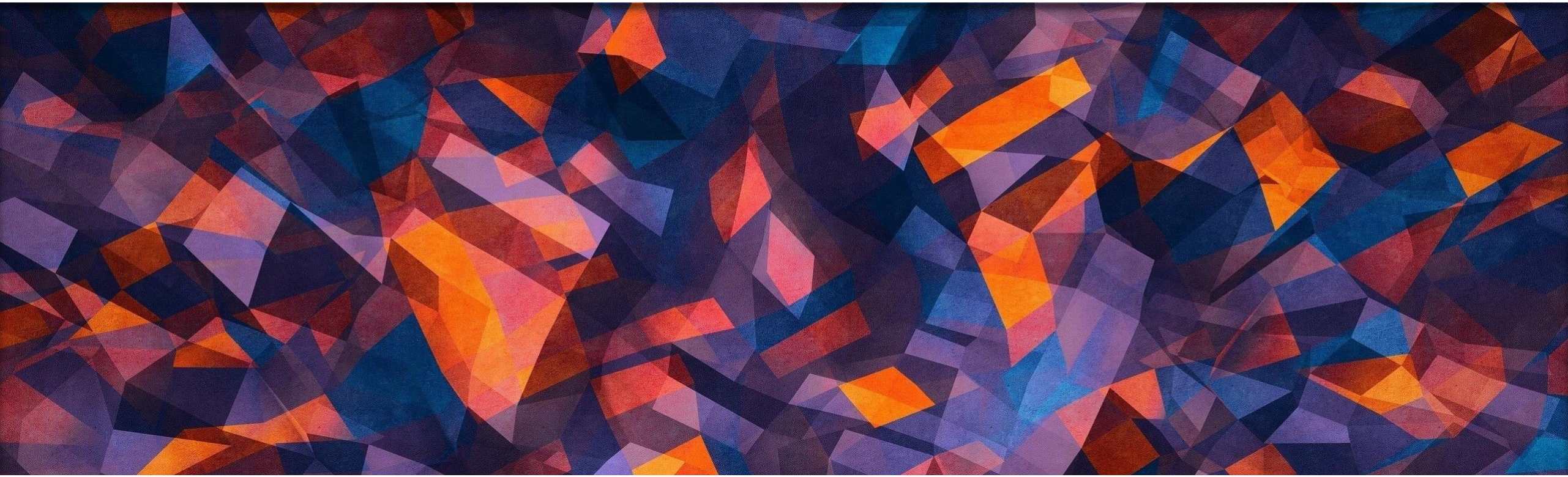
Universität St.Gallen



Event-driven and Process-oriented Architectures

Technology: Distributed Streaming Platform Kafka

Lecture 1

Agenda

- Kafka Core Concepts
- Kafka Producers: Writing Messages to Kafka
- Kafka Consumers: Reading Messages from Kafka

Agenda

- **Kafka Core Concepts**
- Kafka Producers: Writing Messages to Kafka
- Kafka Consumers: Reading Messages from Kafka

Universität St.Gallen

The Distributed Streaming Plaform Apache Kakfa

- **Distributed event streaming platform**, originally developed by LinkedIn; since 2011 open source
- Built to **publish** and **subscribe** to streams of events, including continuous import/export of data from other systems
- Kafka **stores** streams of events **durably** and **reliably** for as long as you want. This distinguishes Kafka from other message-oriented solutions (e.g., RabbitMQ), since events that are published in Kafka are not deleted as soon as they are consumed.
- Streams of events can be **processed** as they occur or retrospectively.
- Above functionality provided in a distributed, highly scalable, elastic, fault-tolerant, and secure manner

Kafka Core Concepts

- Messages and Batches
- Schemas
- Topics and Partitions
- Producers and Consumers
- Clusters and Broker

Messages and Batches in Kafka

- The unit of data in Kafka is called a **message** (also called a **record** or an **event** in a stream processing context)
- For Kafka messages are byte arrays
- Messages can have an optional piece of metadata, called a **key**
- For efficiency reasons messages are written to Kafka in **batches**
  - Batch size: trade-off throughput and latency

Clashing Terms

Means of transport

Message, record, event

Type of payload

Event

Command

Other (Query, document)

Attention: When the term event is used, it might refer to the real event in the payload or the message containing the event

Schemas

- For Kafka messages are opaque byte arrays
- Additional structure (i.e., **schema**) can be imposed on the message content

JSON / XML

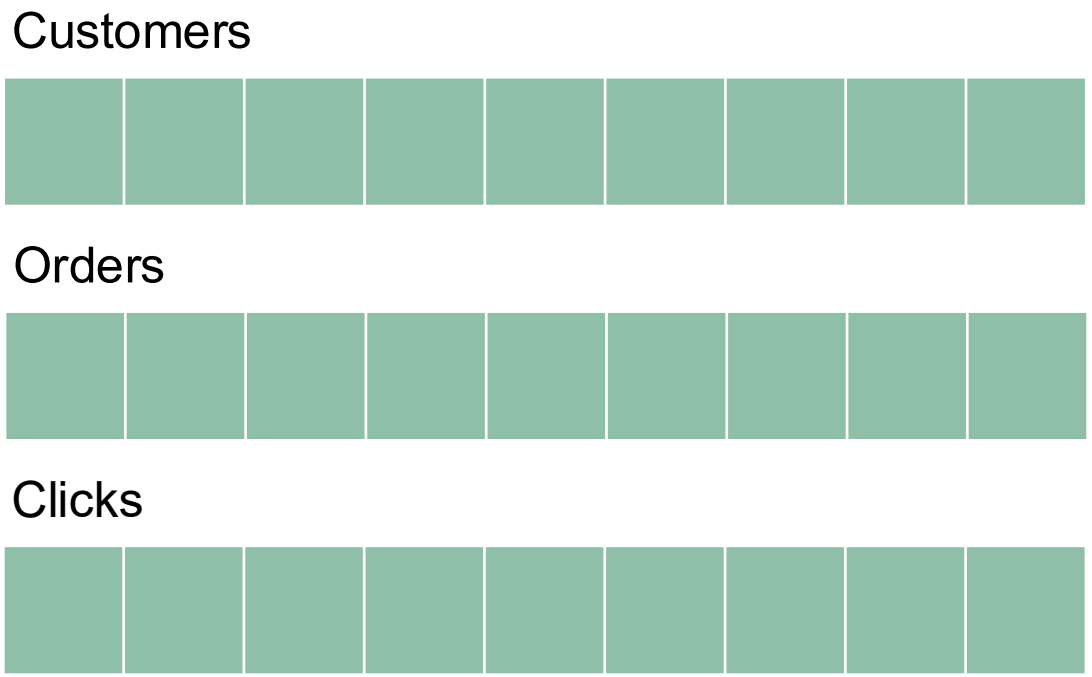
- Easy to use and human readable
- Lack robust type handling and schema

Apache Avro

- Compact serialization format
- Schemas separate from the message payload
- Strong data typing
- Schema evolution with backward and forward compatibility

Topics

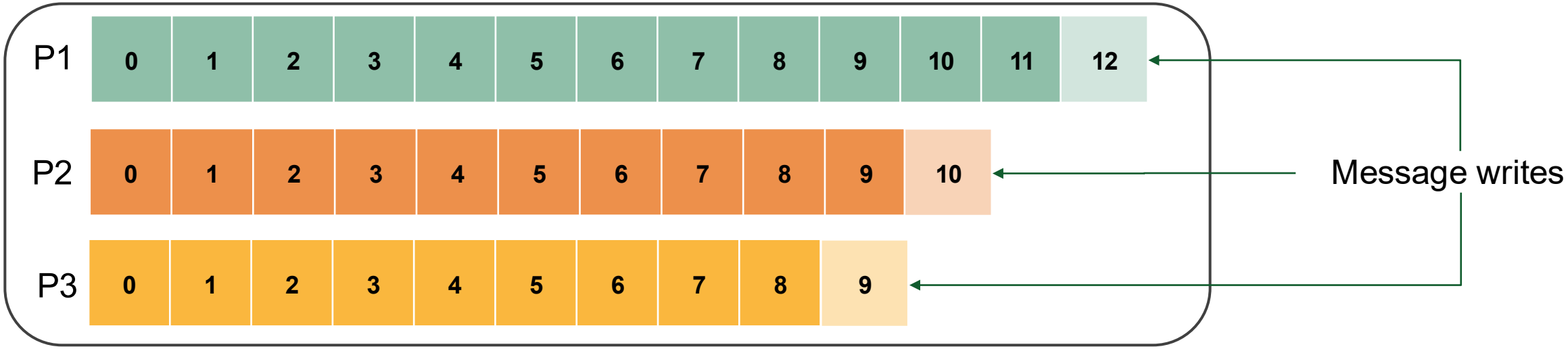
- Messages in Kafka are categorized into **topics**
- A topic represents a named **stream** of events
- Applications **write to** and **read from** topics



Partitions
Structure and Ordering

- Topics are broken down into **partitions**
- Each partition is a single **log**
  - Messages are written to the log in an append-only fashion
  - Messages are read in order from beginning to end
  - Guarantee of message ordering within a partition, but not across partitions

Topic Orders



Why Partitions?
Scalability and Fault Tolerance

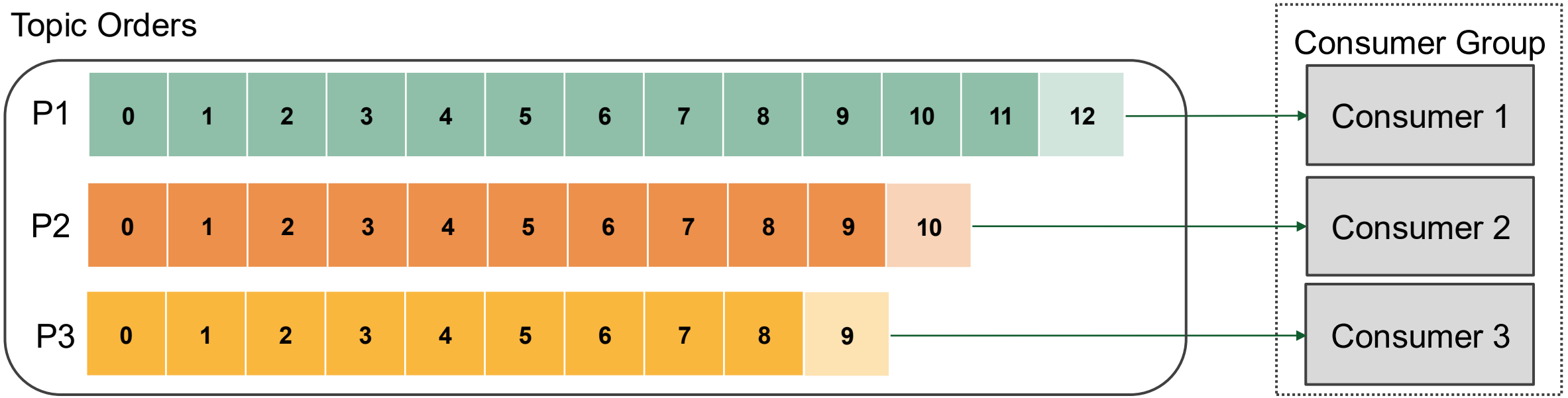
- Partitions are the way how Kafka provides **scalability** and **fault tolerance**
  - Each partition can be hosted on a different broker allowing to scale a single topic horizontally
  - Partitions can be replicated across brokers so another replica can take over in case one fails

Producers and Consumers

- Producers
  - Create new messages to a specific topic
  - Producers can influence the partition a message is written to
- Consumers
  - Read messages
  - Subscribe to one or more topics and read messages in order within each partition
  - Keep track of which messages they already consumed (**offset**)

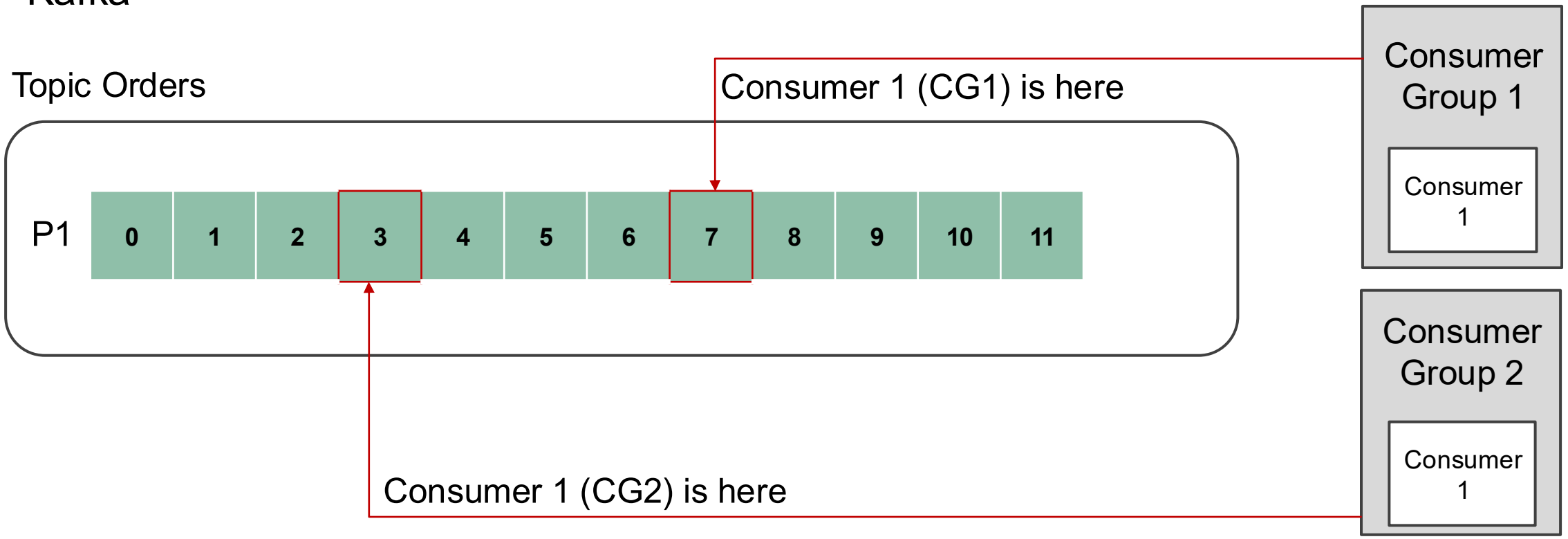
Consumer and Consumer Groups

- Consumers work as part of consumer groups
  - Each partition is only consumed by a single group member (**partition ownership**)
  - Consumers can horizontally scale to consume topics with many messages
  - Consumer groups add fault tolerance; if a single consumer fails, other members can take over



Consumers and Consumer Groups

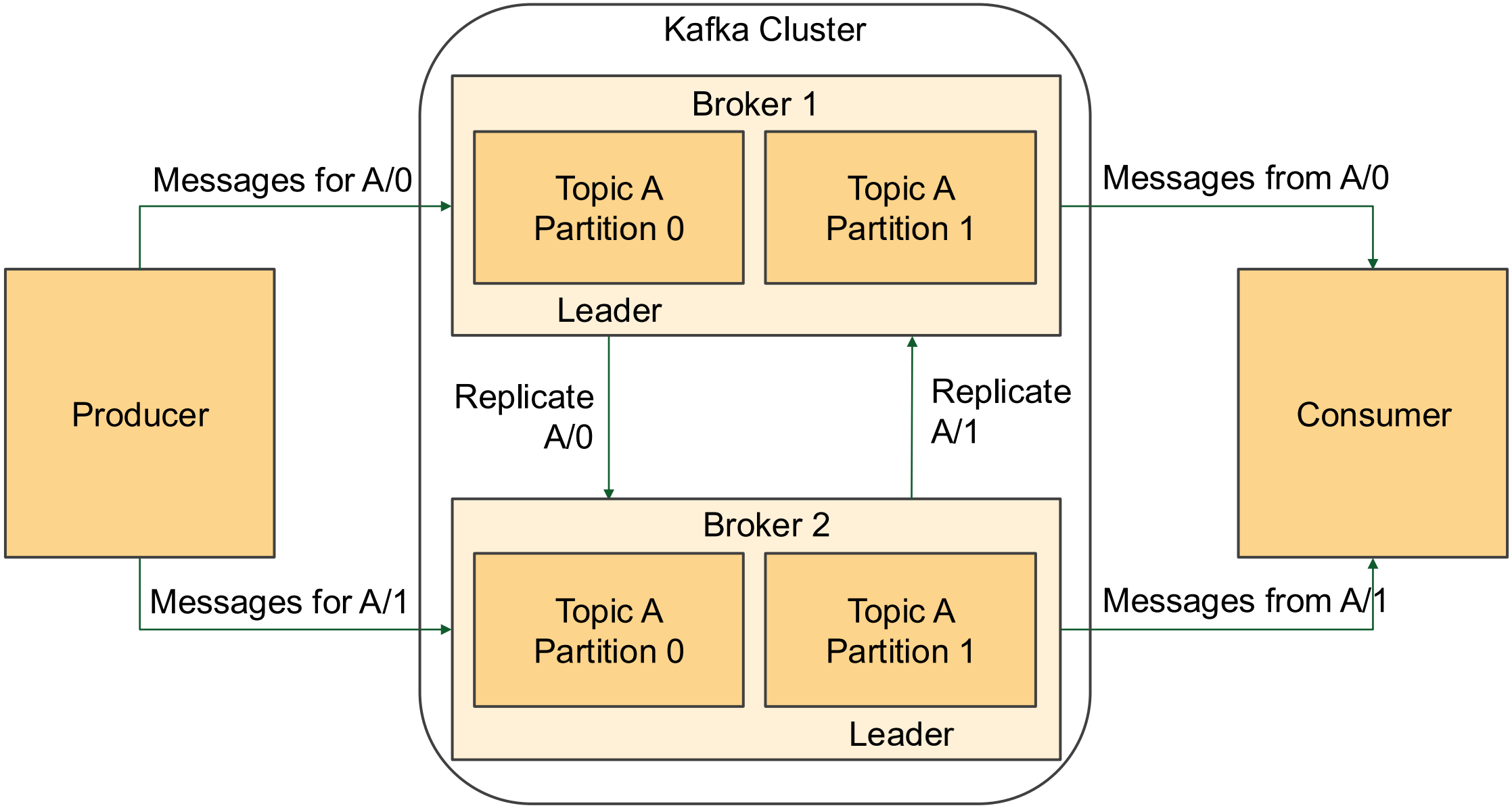
- The same topic can be read by different consumer groups
  - Each message is associated with a unique number (**the offset**) within its partition
  - Each consumer can read messages at its own pace by recording its progress in Kafka



Broker and Cluster

- A single Kafka server is called a **broker**
- Brokers
  - Receives messages from producers, assigns offsets, and stores messages on disk
  - Serves consumers by returning messages from requested partitions
- Kafka brokers operate as part of a **cluster**
- Kafka clusters include a ***controller*** role that manages metadata (in modern Kafka this is handled via KRaft)
- Each partition has:
  - One **leader broker** handling read and write requests
  - One or more **follower replicas** that replicate the reader's data
- Producers write messages to leaders
- Consumers can reader from the leader, and if configured, from followers

Replication of Partitions in a Kafka Cluster



Broker and Cluster
Retention

- Durable storage of messages for a certain period of time
  - Brokers can be configured to retain messages
    - for some period of time (e.g., 7 days) or
    - until partition reaches a certain size in bytes (e.g., 1 GB)
  - Upon reaching these limits messages are expired and deleted

- Topic can be *log compacted*, i.e., only the last message produced with a certain key is retained
  - Useful for changelog-type data

Attention. Be aware of consumer lags

Kafka Use Cases

- Activity Tracking
- Messaging
- Metrics and Logging
- Commit Log
- Stream Processing
- Event Sourcing

Use Case: Activity Tracking

- Original use case by LinkedIn
- User actions → published as event streams
- Consumed for analytics, monitoring, ML

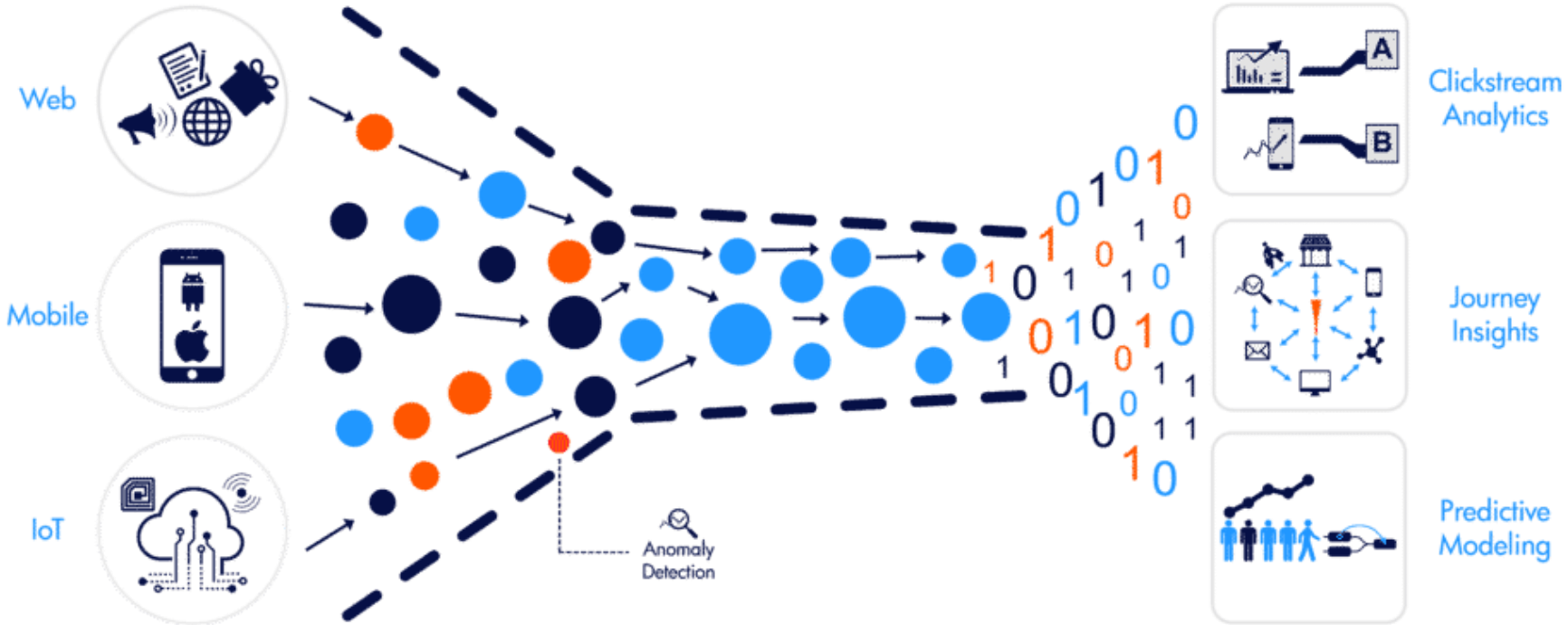
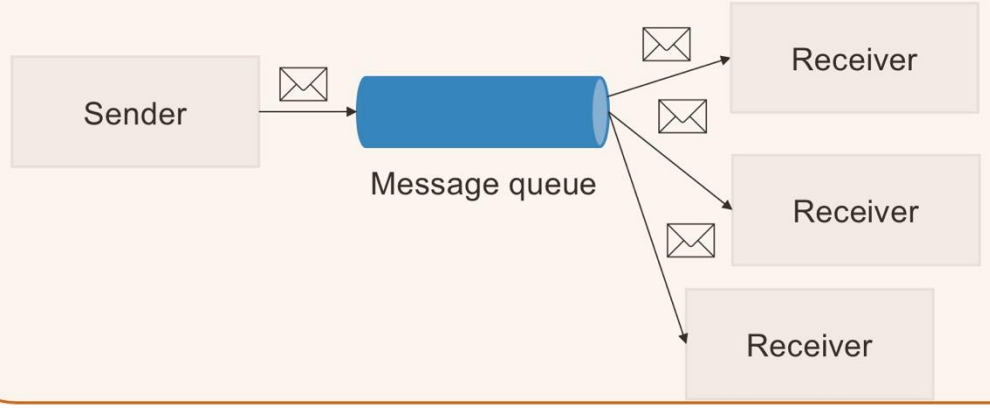


Image Source: https://medium.com/@ugurekmekci/real-time-user-activity-tracking-w-divolte-collector-and-kafka-d8c106313400

Use Case: Messaging

- Point-to-point messaging

- Publish-subscribe messaging



Kafka

- Kafka for messaging
  - can replace traditional message brokers (e.g., ActiveMQ or RabbitMQ)
  - designed for high throughput and scalability
  - Built-in partitioning, replication, and fault tolerance
  - Best suited for large-scale message processing
  - Also useful for low-volume systems that need durability, replay, or multiple consumers

Use Cases

- Operational Observabiltiy
  - Metrics and logging
    - Collect application and system metrics in real time
    - Enable monitoring, alerting, and log search (e.g., Elasticsearch)
    - Decouple data producers from analysis tools

- System Integration
  - Exertnal commit log
    - Kafka can serve as a durable event log for distributed systems
    - Services can replicate state changes and receive live updates
    - Replay enabled recovery after failures or logic changes
    - Log compaction keeps only the latest value per key

Use Cases

- Stream processing
  - Continuously compute results from event streams
  - Examples: counting, filtering, joining, transforming events
  - Implemented using libraries such as Kafka Streams

- Event Sourcing
  - System state is stored as time-ordered sequence of events
  - Kafka can serve as the system‘s durable source of truth
  - Replay enables state reconstruction and new processing logic

Kafka Use Cases

Activity Tracking

Part 1

Messaging

Metrics and Logging

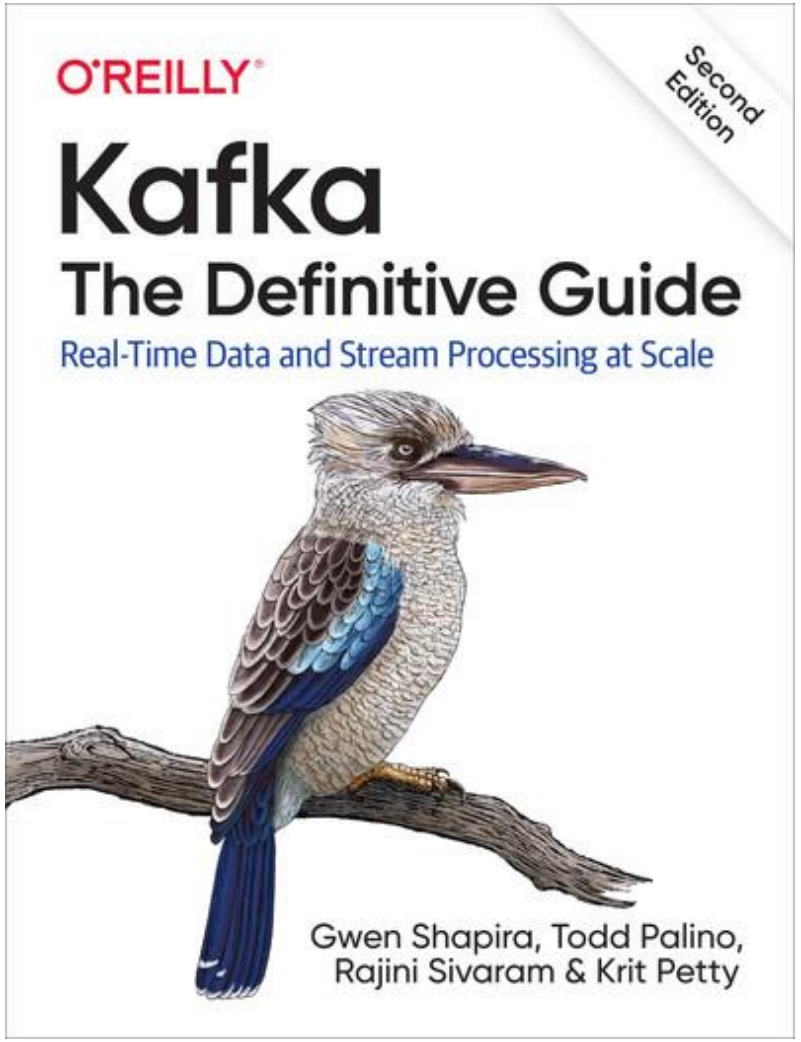
Commit Log

Part 2

Stream Processing

Event Sourcing

References

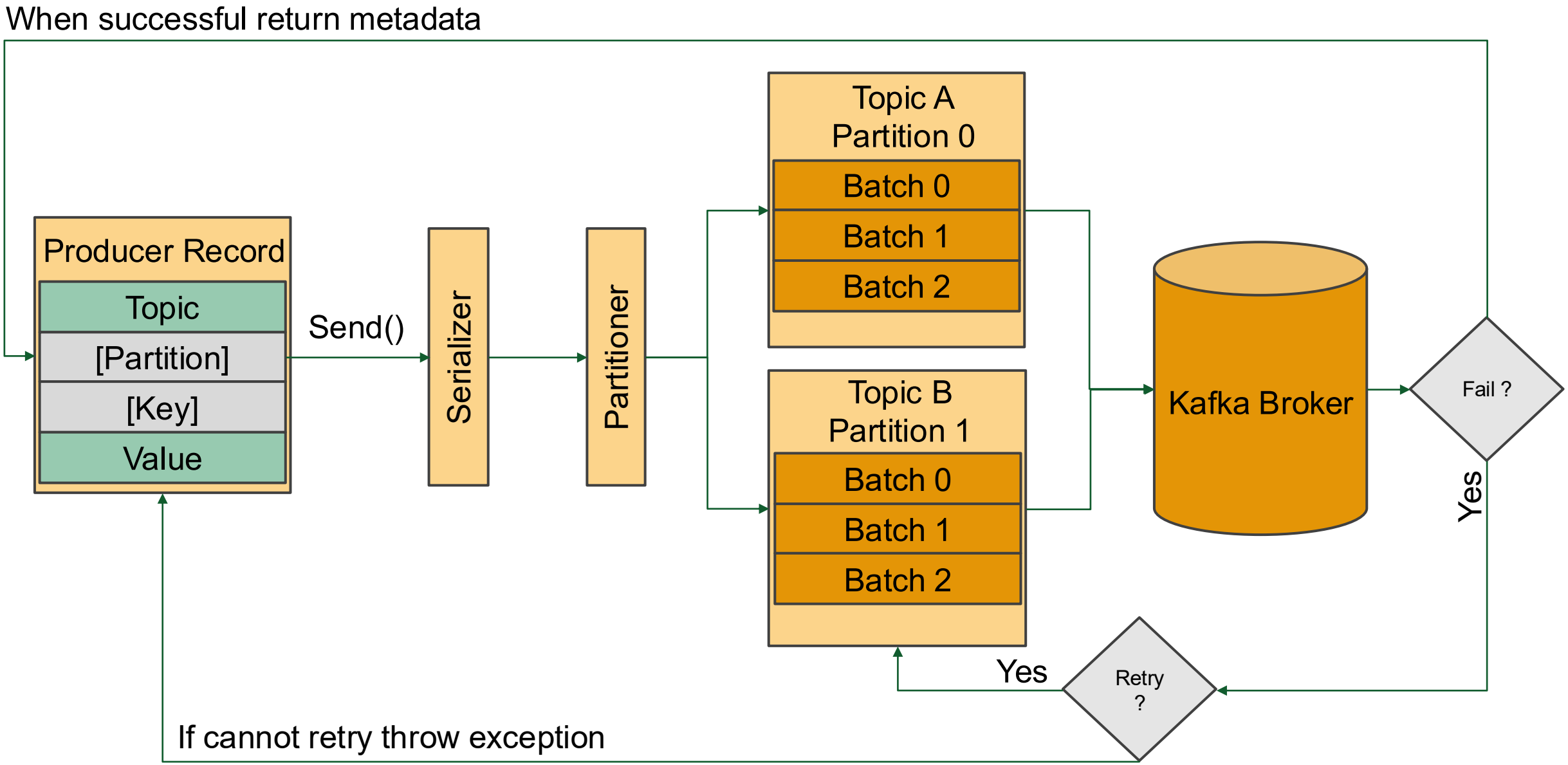


Based upon Chapter 1: Meet Kafka

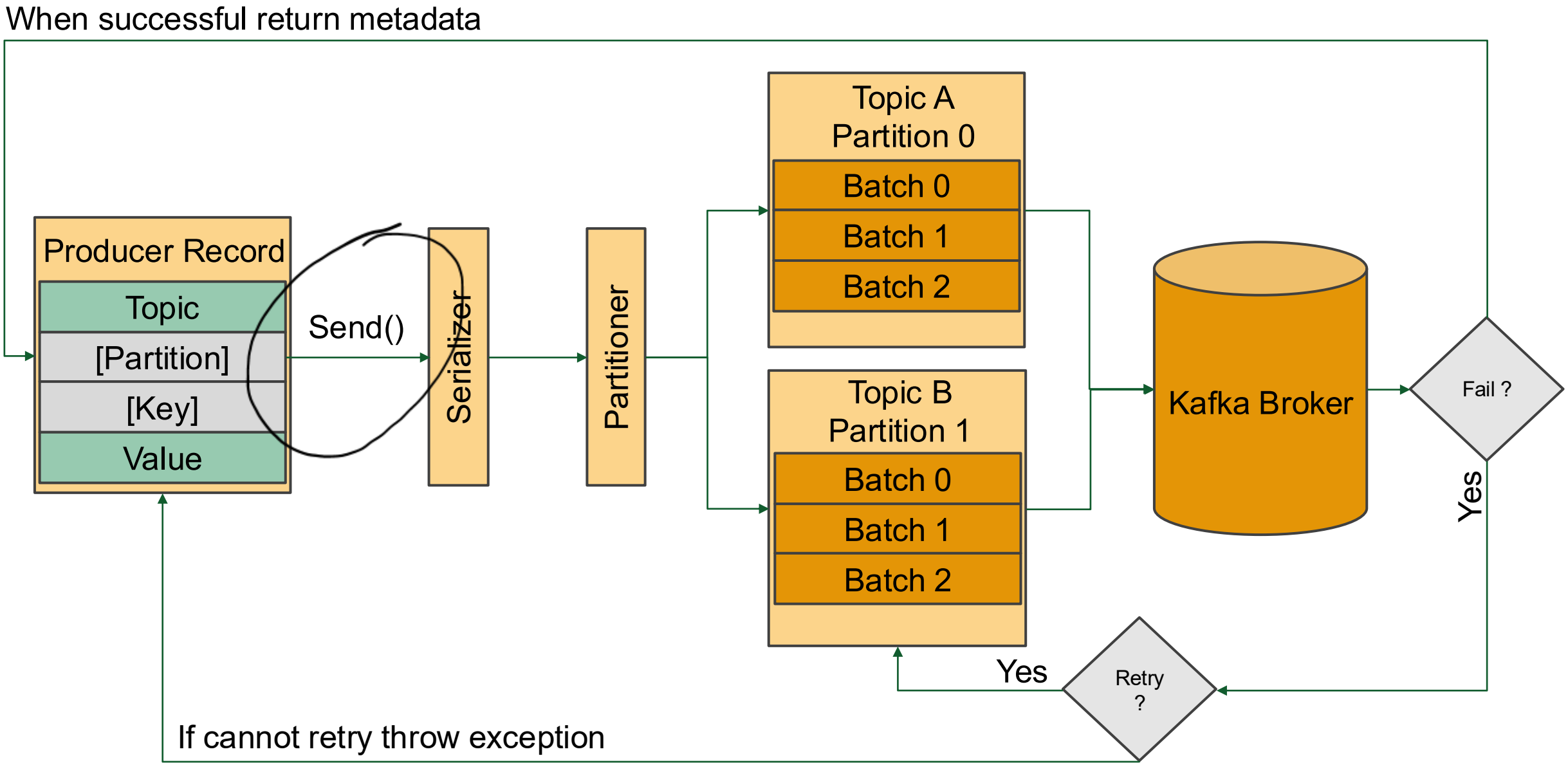
Agenda

- Kafka Core Concepts
- **Kafka Producers: Writing Messages to Kafka**
- Kafka Consumers: Reading Messages from Kafka

Kafka Producer Components
High-level Overview



Kafka Producer Components
High-level Overview



Methods for Sending Messages
Producer Send Patterns

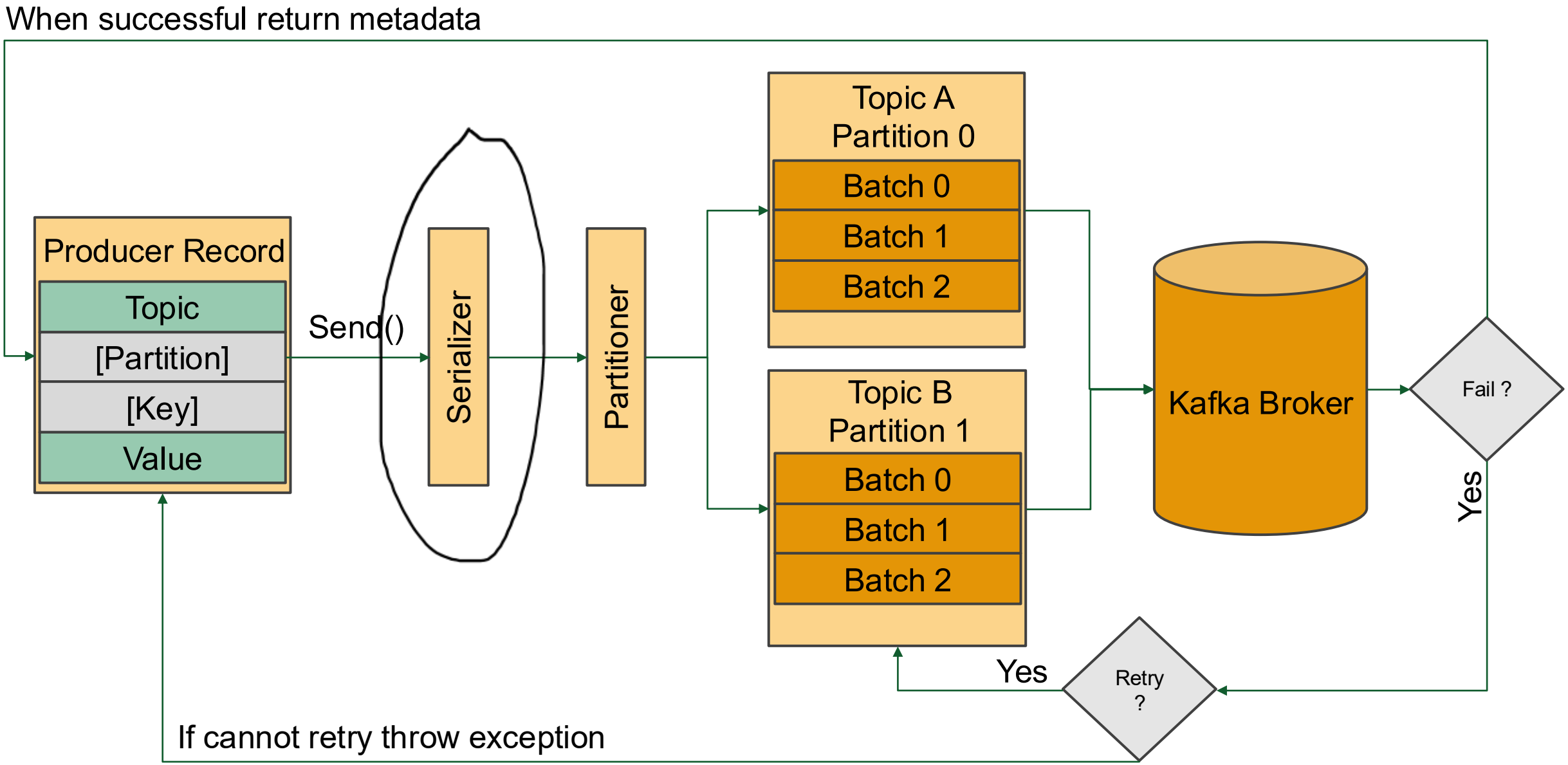
- Fire-and-forget
  - Send message without waiting for confirmation
  - Highest throughput, but message loss may go unnoticed
  - `producer.send(record);`
- Synchronous send
  - Wait for broker acknowledgment before continuing
  - Safer, but blocks the thread → lower throughput
  - `producer.send(record).get();`
- Asynchronous send with callback
  - Send without blocking, handle success or failure in callback
  - Good balance of reliability and performance
  - `producer.send(record, callback);`

Configuring Producers
Acknowledgements acks

- The `acks` parameter controls how many partition replicas must receive the record before the producer can consider the write successful
  - `acks=0`: The producer will not wait for a reply from the broker before assuming the message was sent successful
  - `acks=1`: The producer receives a success response from the broker the moment the leader replica receives the message
  - `acks=all`: The producer receives a success response from the broker once all in sync replicas receive the message

Tradeoff: reliability versus producer latency; no trade off for end-to-end latency since consumers will only be able to read records once they are written to all in sync replicas

Kafka Producer Components
High-level Overview

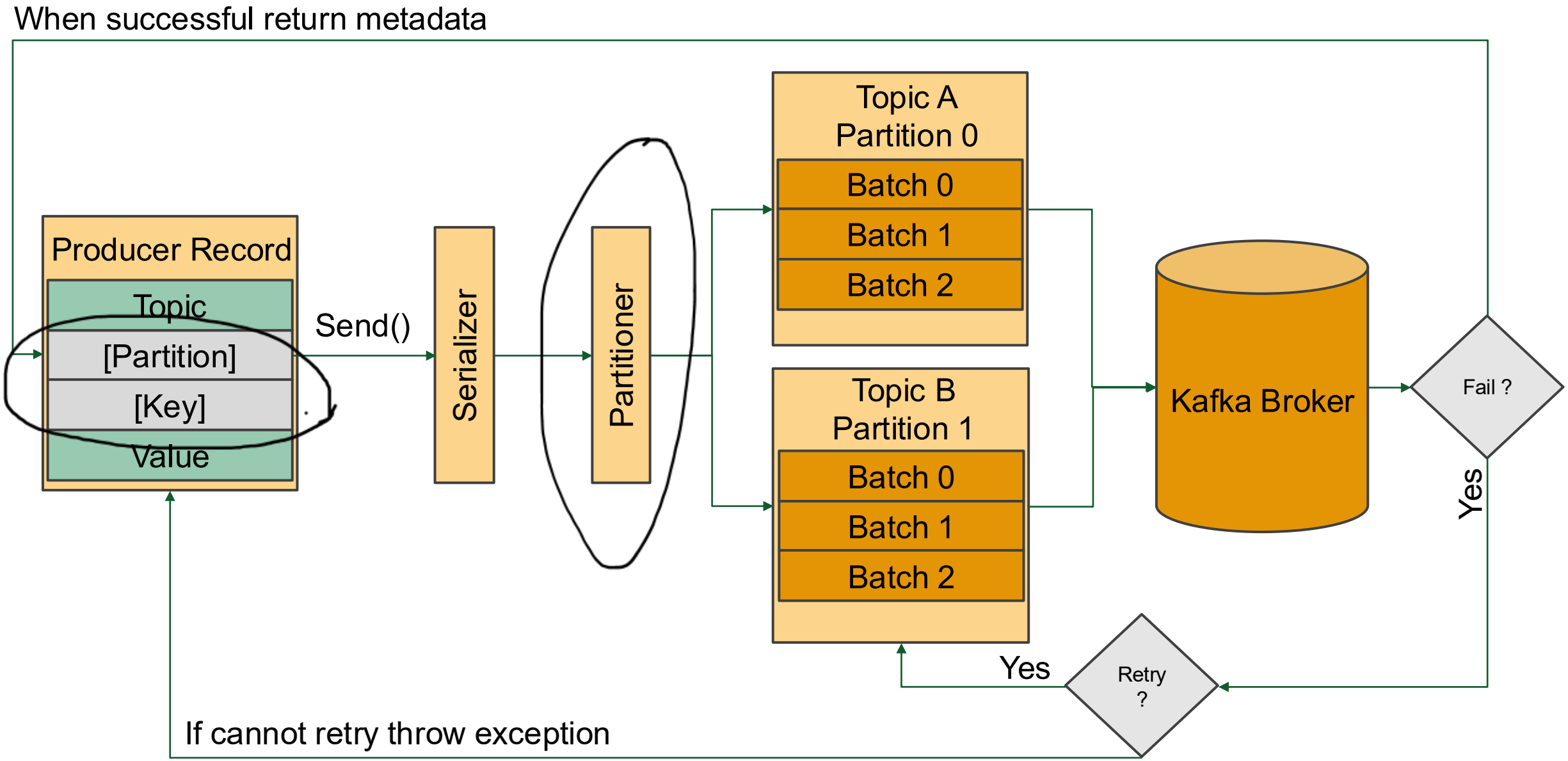


Serializer

Converts application data into bytes for transmission and storage

- Built-in Serializer
  - Provided for common types (e.g., String, Integer, ByteArray)
- Custom Serializer
  - Implemented when no built-in serializer exists
- Schema-based serialization (recommended)
  - Use formats such as Avro, Protobuf, or JSON Schema
  - Enable data compatibility and schema evolution across services

Kafka Producer Components
High-level Overview



Partitioner

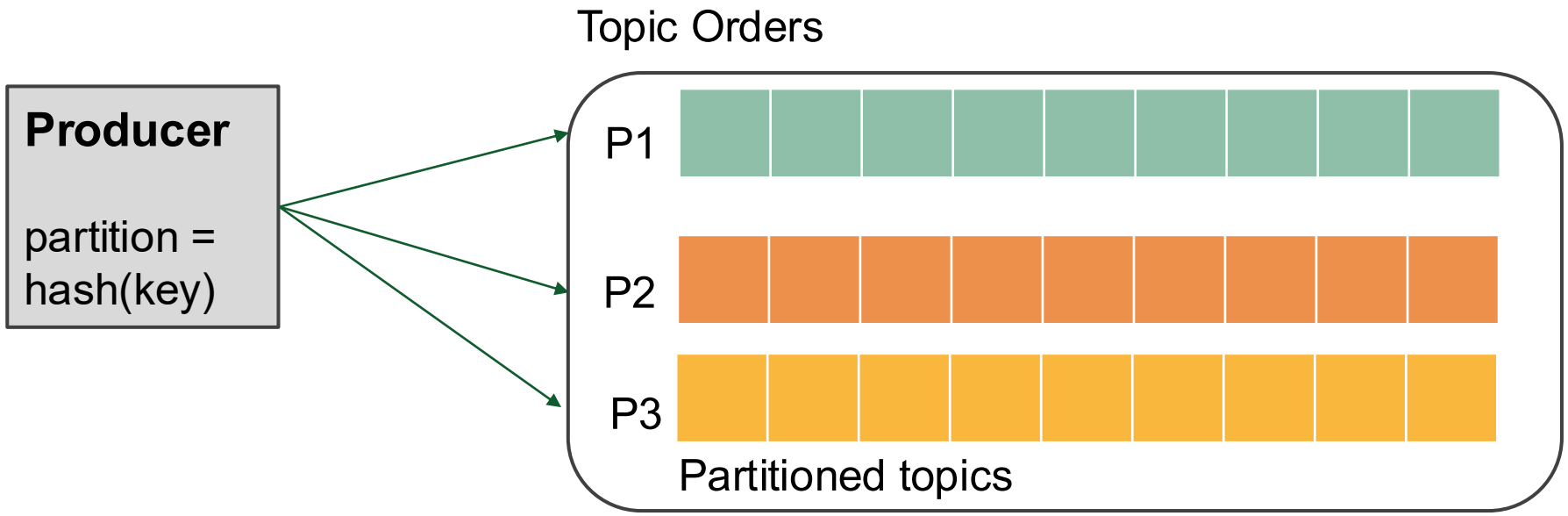
- The event producer can influence the target partition
  - By using a **key**; events with the same key will be written to the same partition
  - When the key is `null` and the default partitioner is used, the message will be sent to one of the partitions at random (sticky round robin)
  - By hardcoding the **target partition number** or specifying **Custom Partitioning logic**

Partitioner

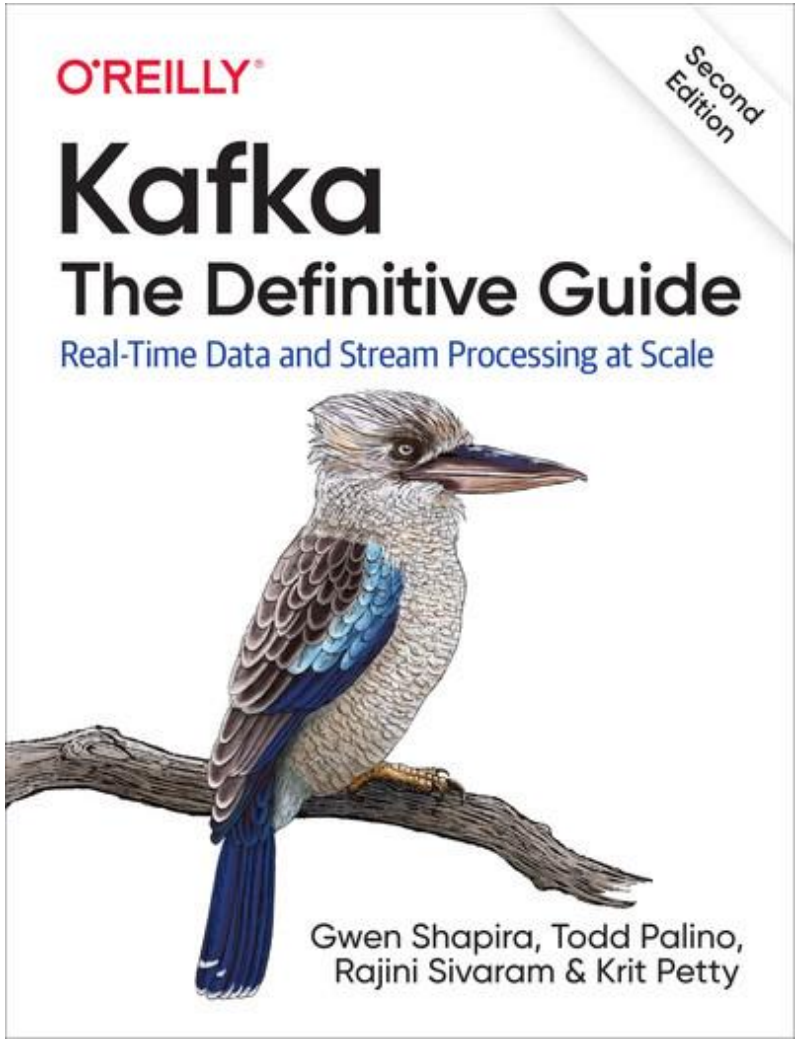
Attention. The mapping of keys to partitions remains stable only while the number of partitions in a topic does not change. If partitions are added later, the mapping may change and ordering per key is no longer guaranteed.

Recommendation.

- Choose a sufficient number of partitions when creating the topic.
- Avoid increasing partitions if key-based ordering is required.



References



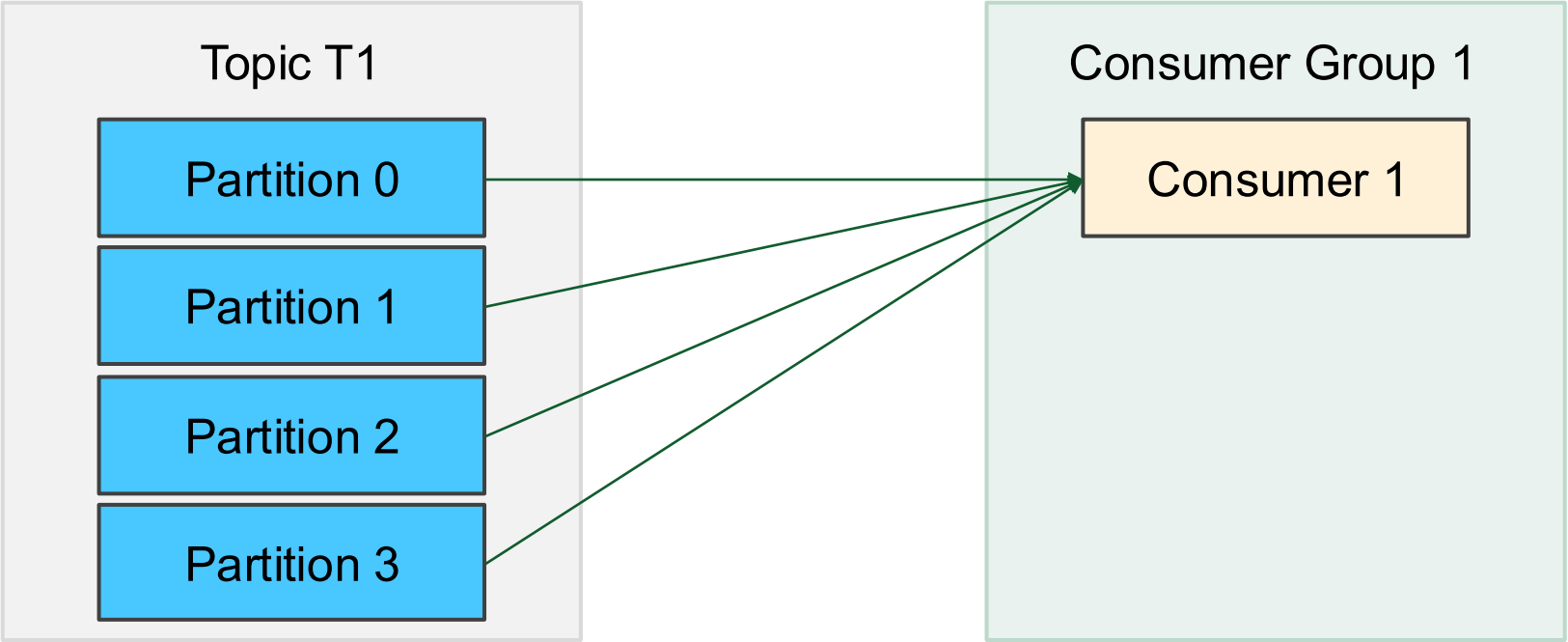
Based upon Chapter 3: Kafka Producers: Writing Messages with Kafka

Agenda

- Kafka Core Concepts
- Kafka Producers: Writing Messages to Kafka
- **Kafka Consumers: Reading Messages from Kafka**

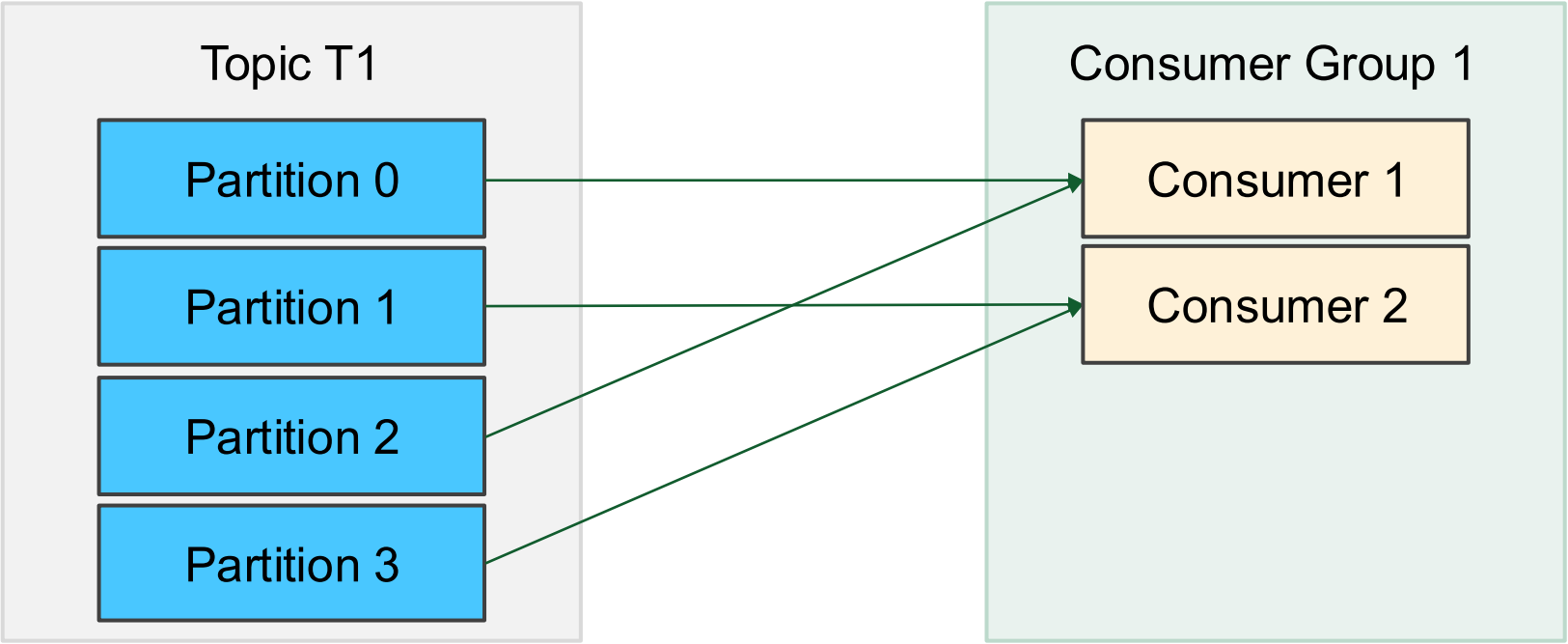
Kafka Consumer Concept
Consumers and Consumer Groups

- **Consumers** are typically part of a **consumer group** to horizontally scale consumption from topics
- Consumers belonging to the same consumer group will consume messages from a different subset of the partitions in the topic



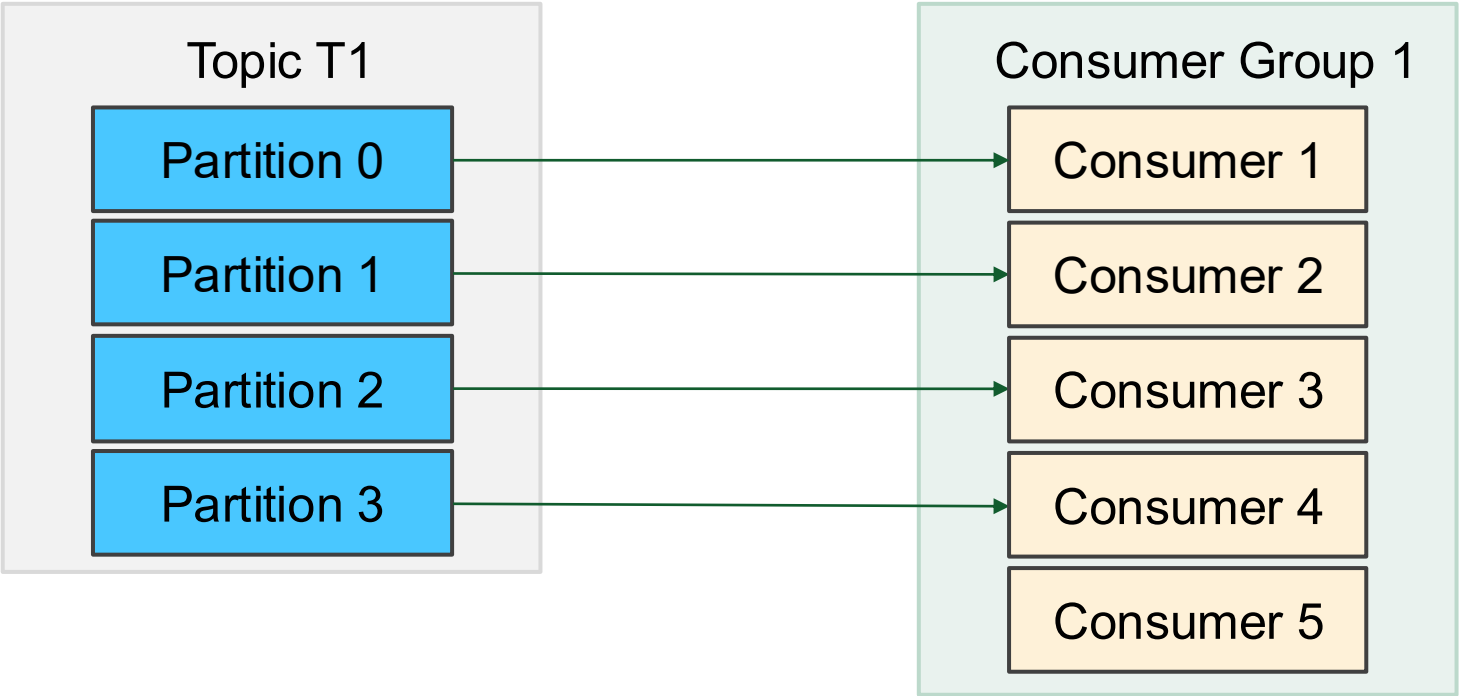
One consumer group with four partitions

Kafka Consumer Concept
Consumers and Consumer Groups



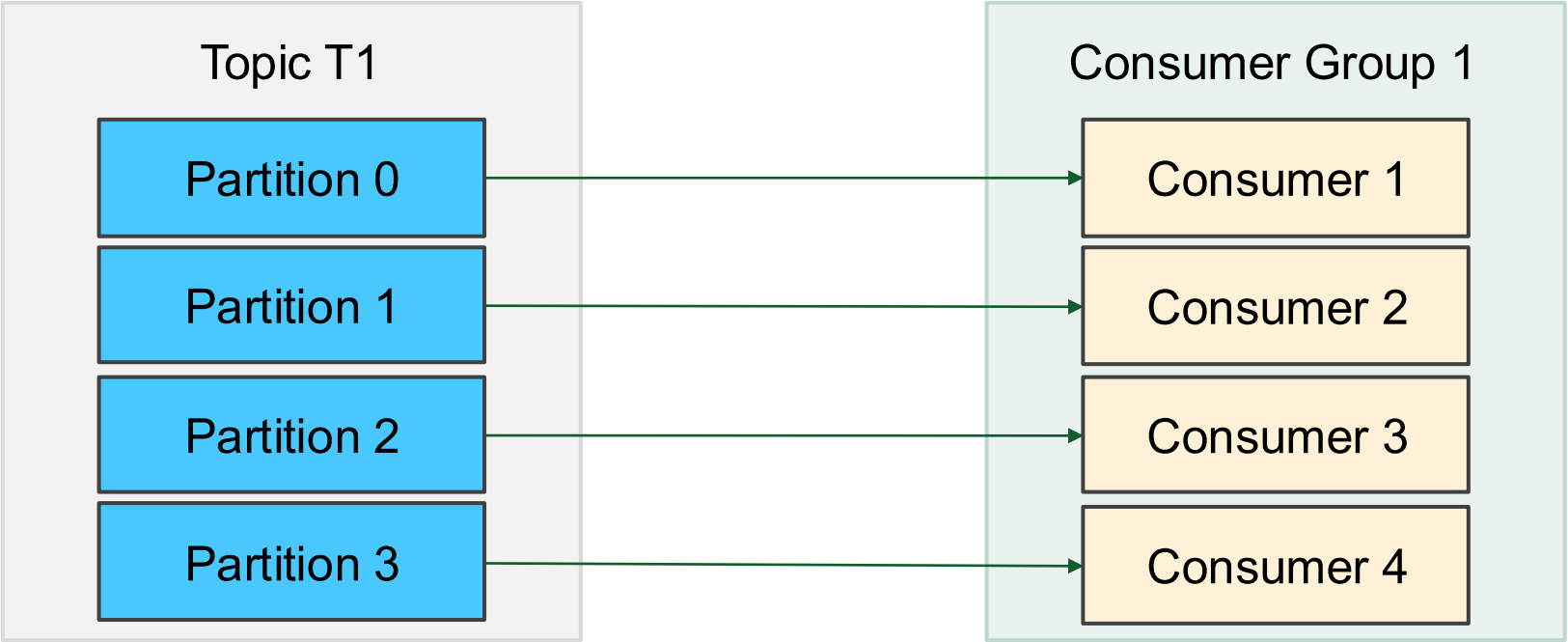
Four partitions split to two consumers in a group

Kafka Consumer Concept
Consumers and Consumer Groups



More consumers in a group than partitions means idle consumption

Kafka Consumer Concept
Consumers and Consumer Groups

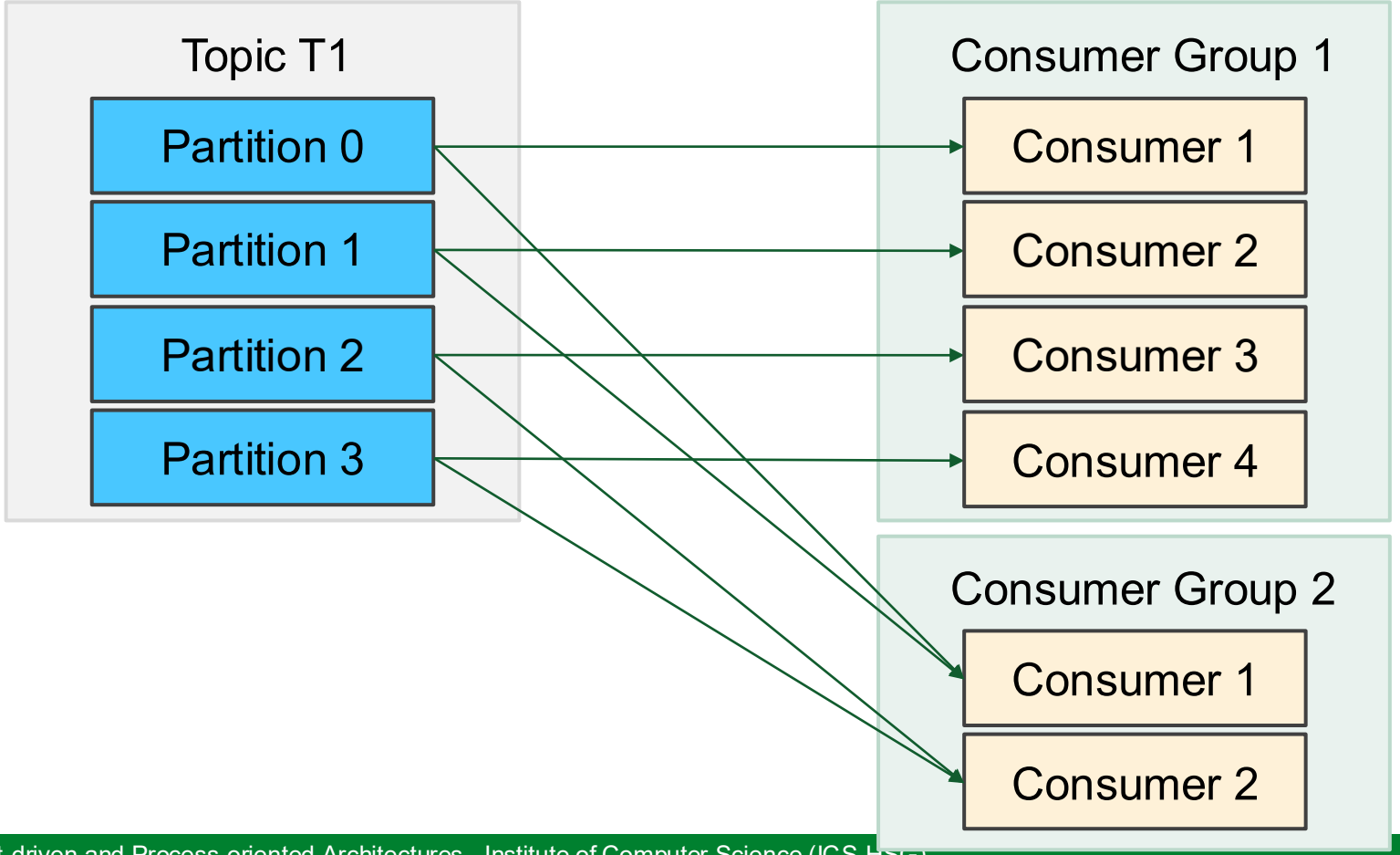


Maximum parallelism = number of partitions.

Four consumers in a group with one partition each

Kafka Consumer Concept
Consumers and Consumer Groups

- We can have multiple applications that need to consume data from the same topic (i.e., multiple consumer groups)

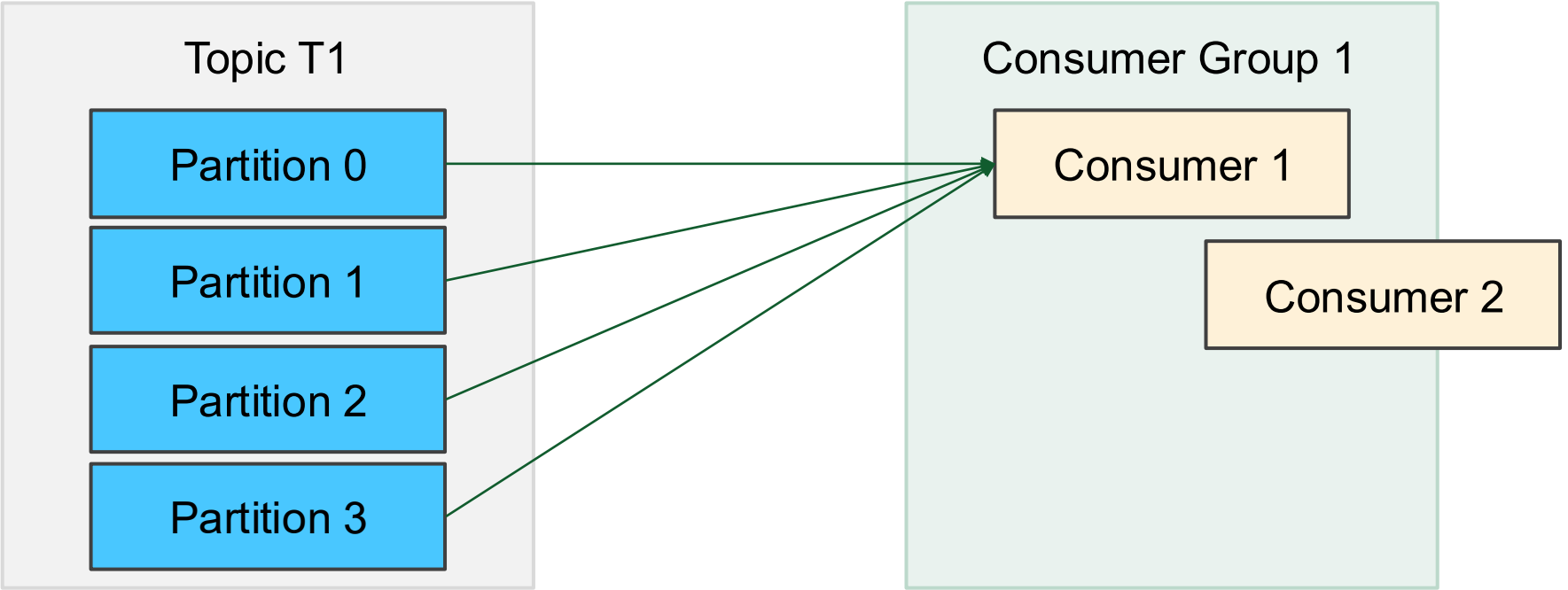


Consumer groups enable independent applications to process the same data.

Adding more consumer groups
All groups receive all messages

Kafka Consumer Concept
Consumer Groups and Partition Rebalance

- When a new consumer joins a consumer group, it begins consuming partitions that were previously assigned to another consumer
- Partition ownership also changes when a consumer fails or when the number of partitions is modified
- **Rebalancing** is the process of reassigning partitions among consumers in the group
- During a rebalance, message consumption pauses briefly until reassignment is complete



Consumer Groups – Key Rules

- Parallelism limited by number of partitions
- One partition → one active consumer per group
- Multiple groups → independent processing of same data
- Rebalance → temporary pause, then reassignment

Kafka Consumer Basics

- Consumers join a consumer group and subscribe to topics
- Messages are retrieved using a continuous poll loop
- If a consumer stops polling, it is considered failed and partitions are reassigned (rebalance)

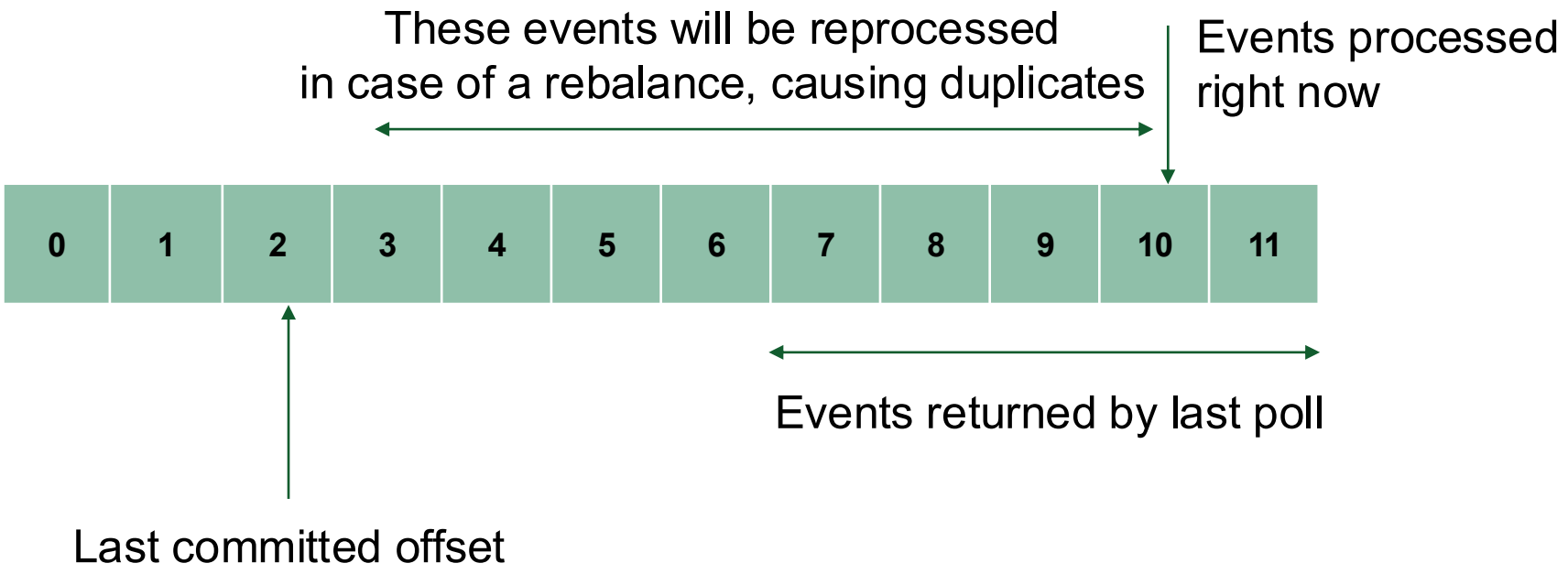
Consumption in Kafka is pull-based and group-coordinated.

Kafka Consumer Concept
Offsets and Commits

- Each call to `poll()` returns messages that have not yet been read by consumers in the same consumer group
- Updating the current position in a partition is called `offset commit`
- After a rebalance, a consumer continues reading from the
  - Last committed offset of each partition
  - If the committed offset is smaller than the last processed message → messages are reprocessed (duplicates)
  - If the committed offset is larger than the processed messages → messages get lost

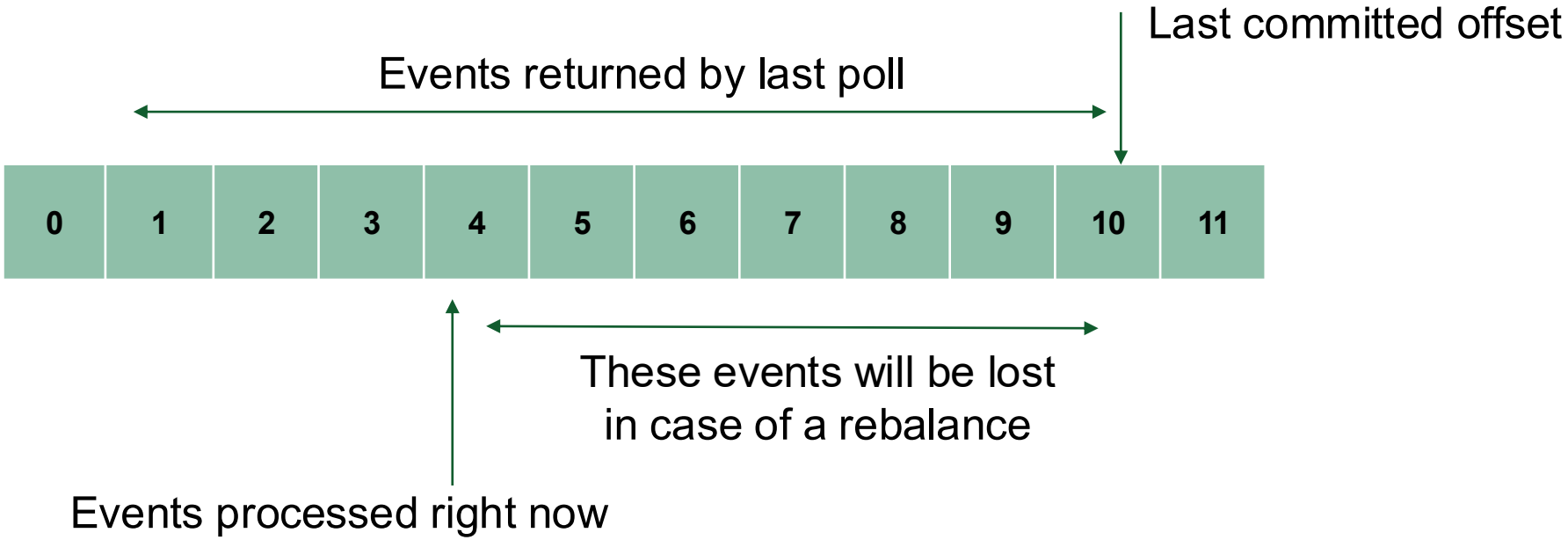
Kafka Consumer Concept
Offsets and Commits – Reprocessing Messages

- After a rebalance, the consumer resumes from the latest committed offset of each partition
- If the committed offset is behind the last processed message → some events are processed again (duplicates)



Kafka Consumer Concept
Offsets and Commits – Missed Messages

- After a rebalance, the consumer resumes from the latest committed offset of each partition
- If the committed offset is ahead the last processed message → some events are skipped (data loss)



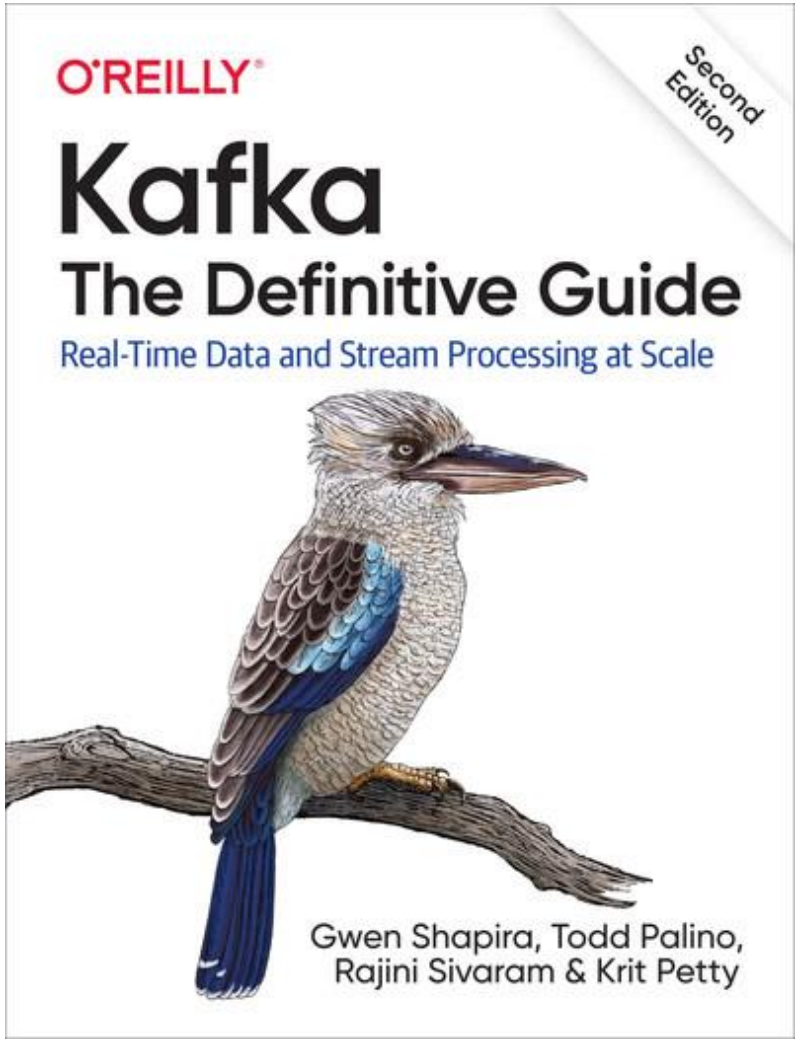
Kafka Consumer Concept
Offsets and Commits

- Automatic Commit
  - Offsets are committed periodically
  - Simple to use, but may lead to duplicate processing
- Synchronous Manuel Commit
  - Offsets are committed after processing is complete
  - Safest option, but introduces additional latency
- Asynchronous Manuel Commit
  - Offsets are committed without blocking
  - Higher performance, but commit failures must be handled

Kafka Consumer Concept
Delivery Semantics

- **Commit after processing** → at-least-once delivery (duplicates possible)
- **Commit before processing** → at-most-once delivery (message loss possible)
- **Exactly-once** delivery requires additional mechanisms (covered in later lectures)

References and Further Materials

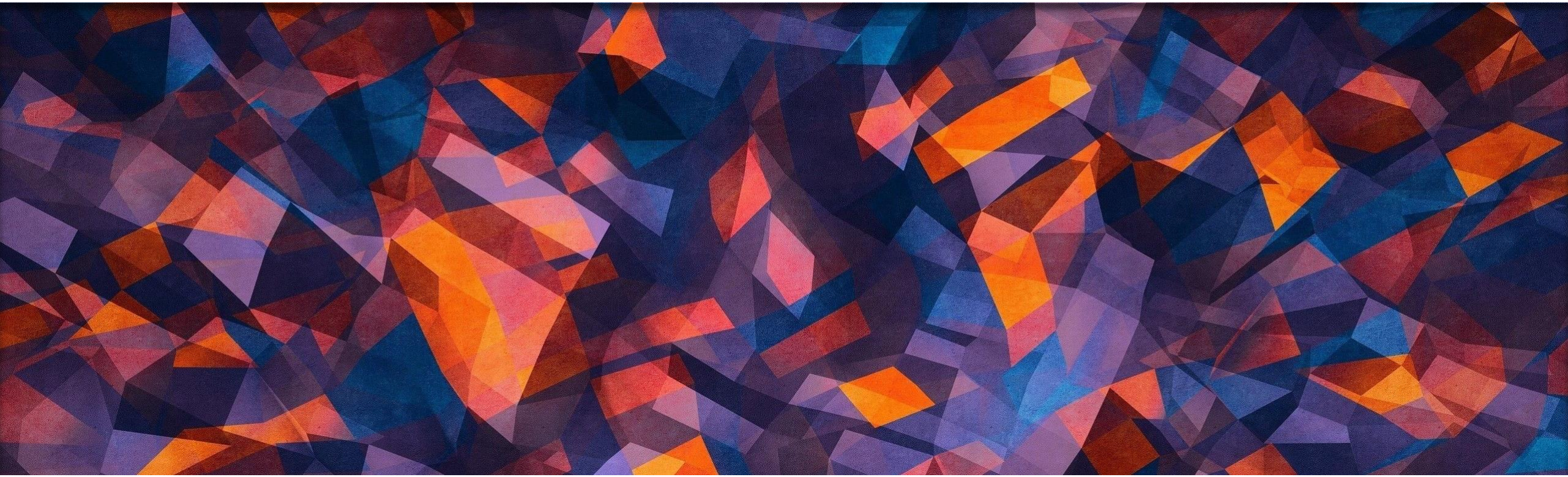


Based upon Chapter 4: Reading Data from Kafka

Further Materials:

- Optimizing Kafka Producer: https://strimzi.io/blog/2020/10/15/producer-tuning/
- Optimizing Kafka Consumer: https://strimzi.io/blog/2021/01/07/consumer-tuning/

Universität St.Gallen



Event-driven and Process-oriented Architectures

Exercise

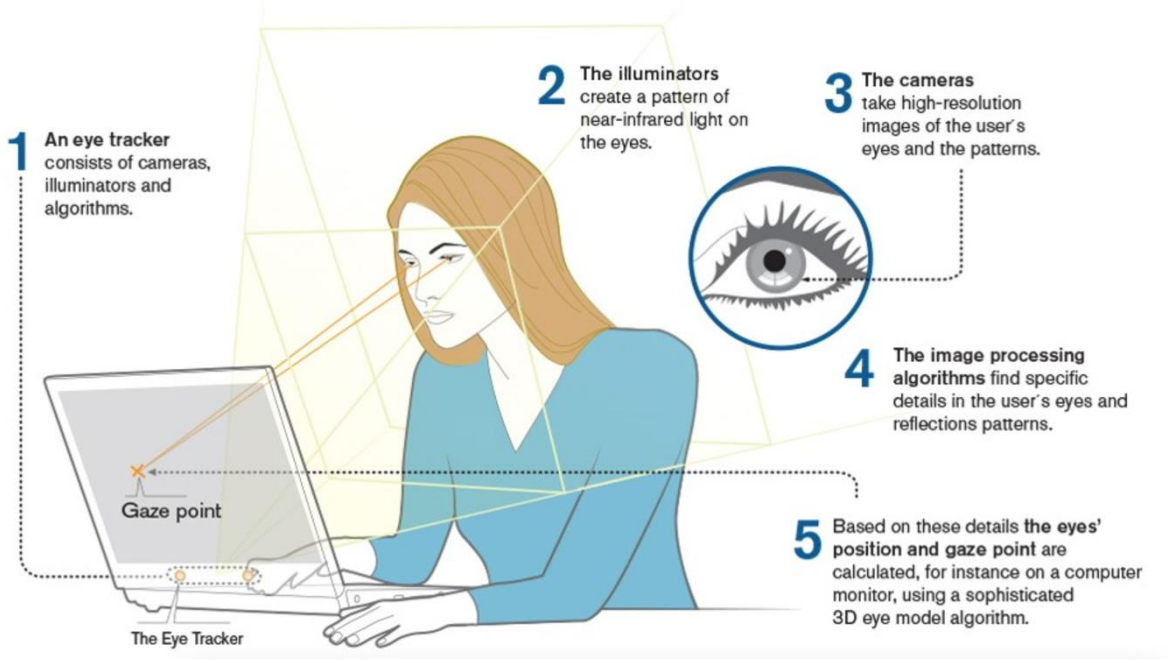
Lecture 1

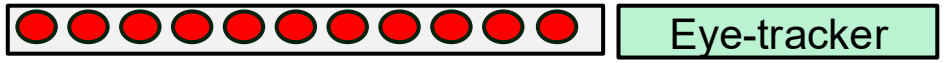
Lab Exercises

- Lab1: Kafka Getting Started
  - https://github.com/scs-edpo/lab01-kafka-getting-started
- Lab2Part1: Simple Producer and Consumer
  - https://github.com/scs-edpo/lab02Part1-kafka-producer-consumer
- Lab2Part2: Producer and Consumer of Eye Tracking Data
  - https://github.com/scs-edpo/lab02Part2-kafka-EyeTracking
- Lab2Part3: Kafka with Multiple Brokers
  - https://github.com/scs-edpo/lab02Part3-ManyBrokers

(Screen-based) Eye-tracking

Lab2Part2





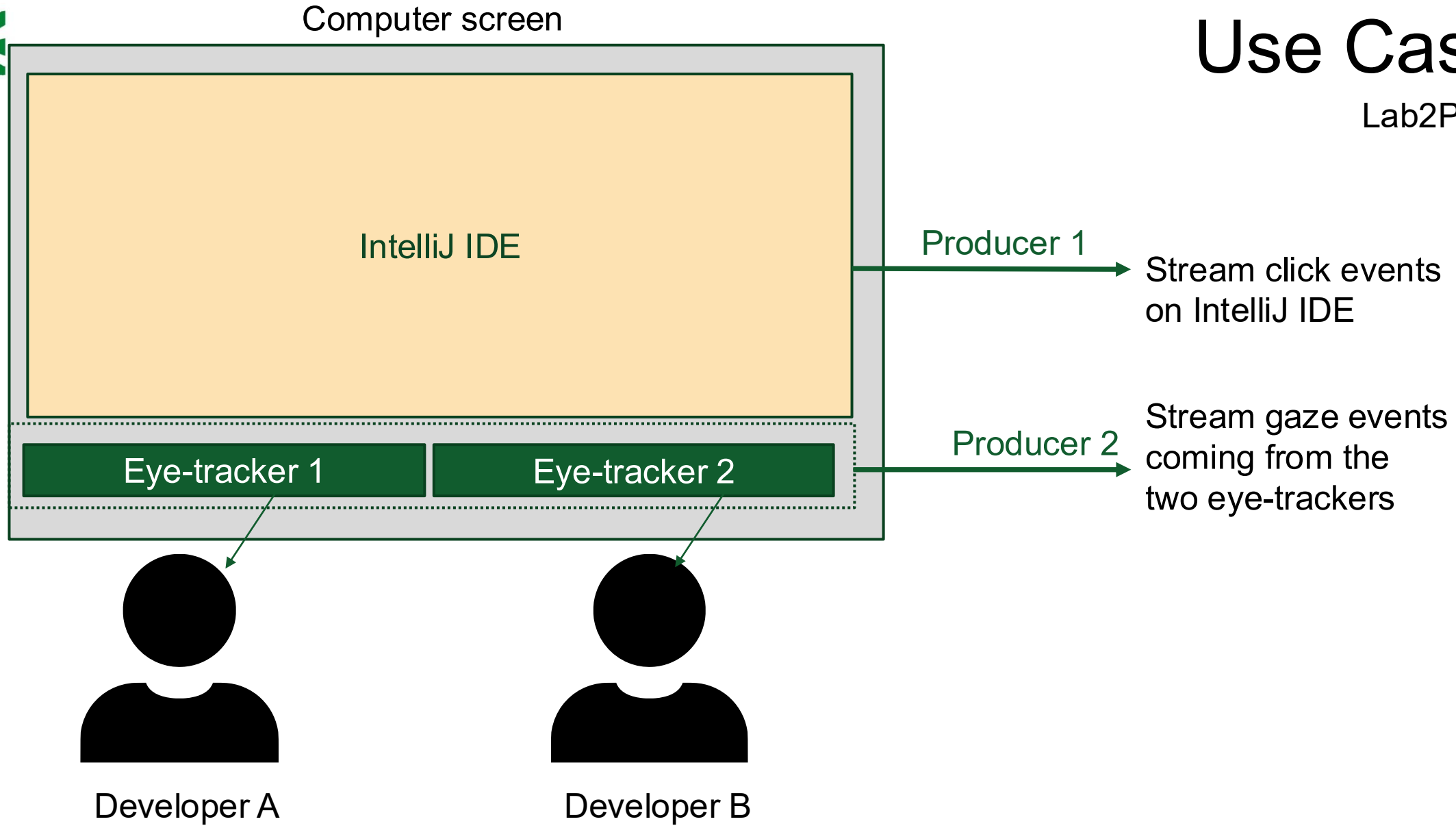
Stream of gaze events

Figure: Screen-based eye-tracking setting

https://connect.tobii.com/s/article/How-do-Tobii-eye-trackers-work?language=en_US

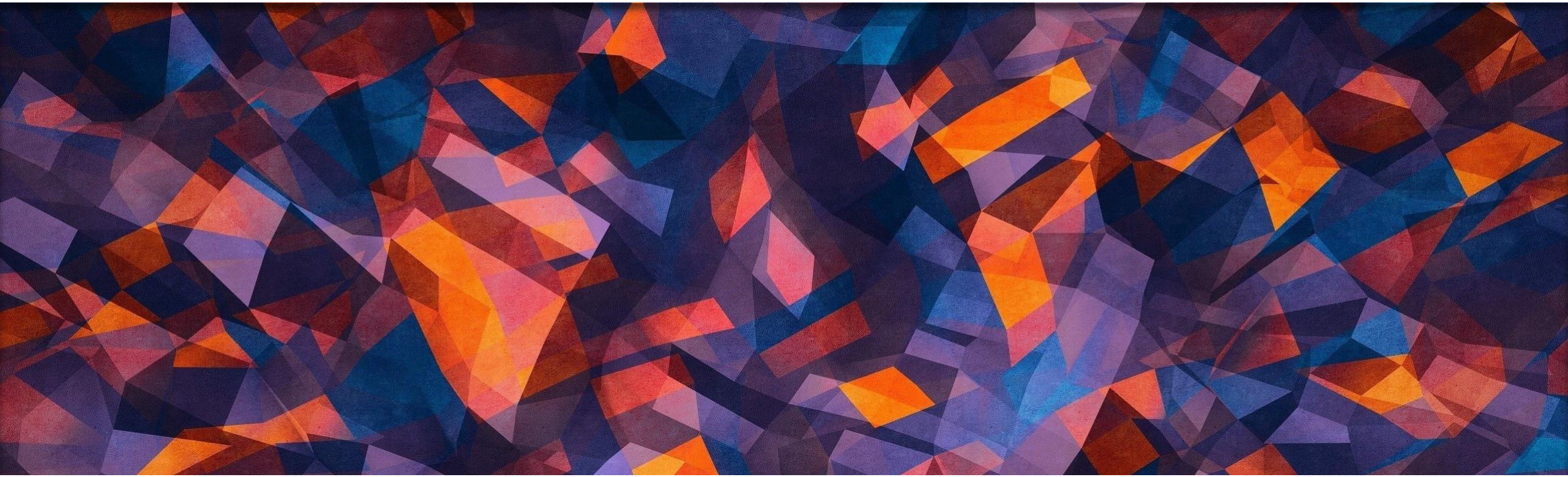
Use Case

Lab2Part2



Two developers doing pair programming

Universität St.Gallen



Event-driven and Process-oriented Architectures

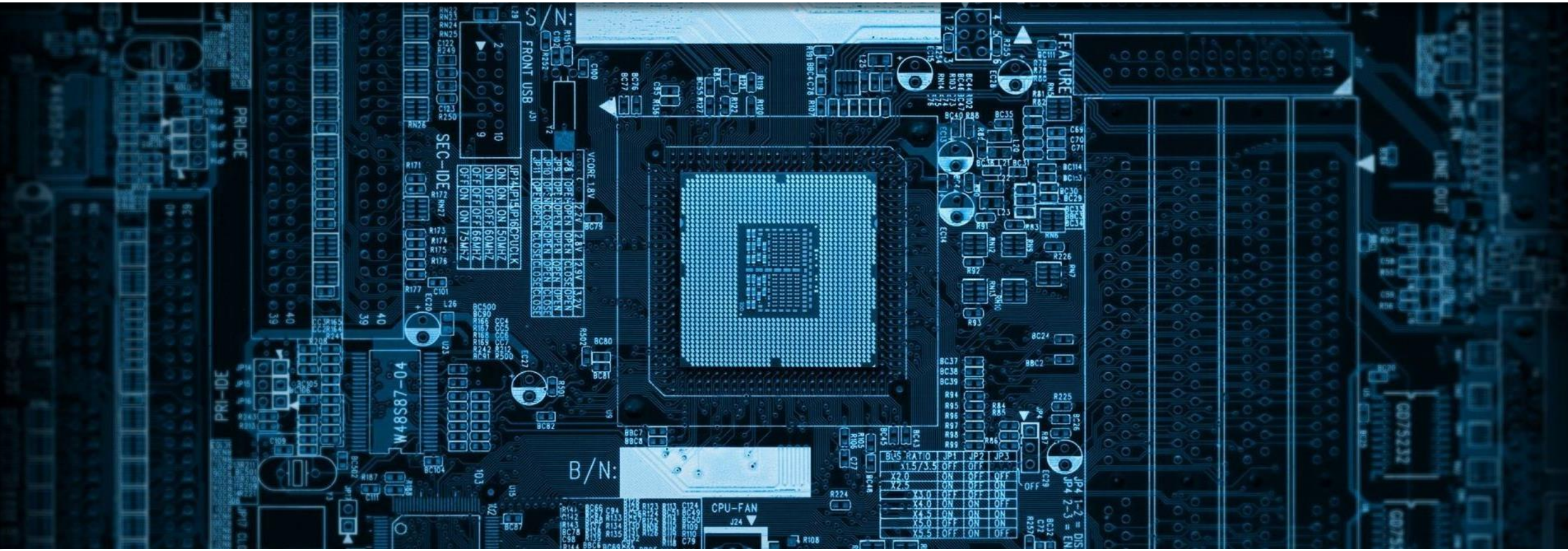
Ideas for Software Project

Lecture 1

Suggestions for Project

- Experiment with Kafka and report on the results
  - Producer Experiments
  - Consumer Experiments
  - Fault-tolerance & Reliability

  See exercise sheet for Exercise 1 on Canvas for more details and suggestions.



Questions?